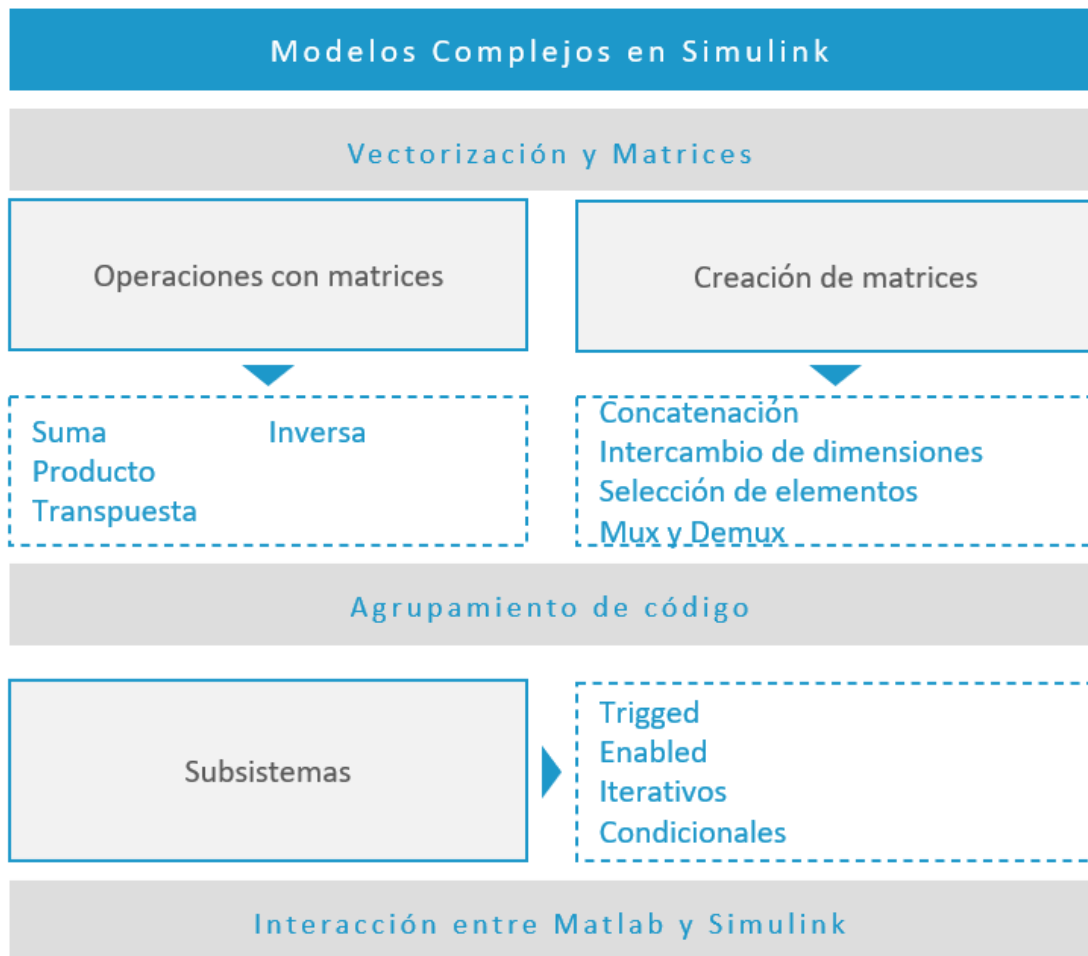


Modelado de Sistemas Dinámicos

Modelos Complejos en Simulink

Índice

Esquema.	2
Ideas clave	3
6.1 Introducción y objetivos	3
6.2 Vectorización y trabajo con Matrices	3
6.3 Agrupamiento de código	7
6.4 Usando Simulink desde Matlab	14



6.1 Introducción y objetivos

En temas anteriores hemos visto como implementar un modelo físico en Simulink y cómo realizar simulaciones con él. En este tema nos centraremos en algunos conceptos avanzados que nos permitirán por un lado optimizar el código de forma que se ejecute de forma más eficiente y por el otro realizar un código más comprensible.

En este tema veremos tres temas fundamentales en la optimización de Simulink:

- ▶ Empezaremos comentando la importancia de la vectorización. Sobre cómo emplear la potencia de Matlab en el trato de las variables puede afectar a la resolución de nuestro código.
- ▶ Continuaremos con la introducción de los sub-modelos. Un sub-modelo consiste en una modelo que tiene unas entradas y salidas y realiza una tarea determinada. Esto nos permite simplificar nuestro modelo y hacerlo más comprensible. Además, esto nos permitirá intercambiar diferentes opciones del modelado en el modelo.
- ▶ Finalizaremos con el tema con estudiando la interacción entre Simulink y el Workflow de Matlab. Esto nos permitirá utilizar modelos de Simulink en Matlab incluyendo cambios en las variables del Workflow y pasándolos a Simulink.

6.2 Vectorización y trabajo con Matrices

Uno de los aspectos fundamentales de Matlab y que se debe tener en cuenta siempre que se trabaje con vectores es la **vectorización**. La vectorización consiste en aplicar ciertas funciones u operaciones directamente sobre vectores y no sobre cada uno de los elementos.

Para visualizar este factor de impacto podemos emplear las funciones `tic` y `toc`. Estas funciones inicializan un reloj externo que nos permite medir el tiempo que transcurre

entre la llamada a `tic` hasta la llamada a `toc`. Un claro ejemplo de este punto de trabajo se puede ver compilando el siguiente código:



Ejemplo de vectorización

```
tic
x=zeros([3,1.e6]);
for i=1:1e6
    x(i)=2*pi/1.e6;
    s(i)=sin(x(i));
end
toc

tic
x=2*pi*(1:1e6);
s=sin(x);
toc
```

Al ejecutar este código obtenemos los resultados:

Elapsed time is 0.142993 seconds.

Elapsed time is 0.088624 seconds.

Así pues, utilizando el bucle `for` consumimos casi el doble de tiempo que cuando utilizamos el código vectorizado.

Vamos a ver los bloques y elementos que nos permiten realizar la vectorización en Matlab y vamos a ver cómo configurar algunos bloques que ya hemos visto para trabajar de la forma adecuada.

Vectores y Matrices

El primer punto a tener en cuenta cuando hablamos de vectorización consiste en trabajar teniendo en cuenta que nuestras señales pueden ser vectores y/o matrices y no únicamente escalares. Así pues podemos:

- ▶ Definir una constante como un **vector** y/o una **matriz** y no únicamente como un escalar. Si definimos la constante como un vector debemos tener en cuenta que Simulink lo interpretará como una matriz columna.
- ▶ Realizar **sumas y productos por escalares**. Utilizando los bloques de `sum` y `product` podemos calcular la suma y el producto por un escalar.
- ▶ Realizar **productos de matrices y matrices por vectores**. El bloque `product` nos permite realizar el producto de dos matrices. Sin embargo deberemos configurarlo para tal efecto. Realizando una doble pulsación sobre el bloque, se nos abrirá la configuración. Deberemos cambiar la opción *Multiplication* que por defecto está en *element-wise(.*)* a *Matrix (*)*. De esta forma podemos multiplicar dos matrices o una matriz y un vector. Deberemos tener en cuenta siempre que las dimensiones de la matriz deben concordar. Además, deberemos tener en cuenta que el producto de matrices no es conmutativo, por lo que el orden de las entradas será importante. Si lo dejamos configurado con la configuración por defecto, entonces realizaremos el producto elemento a elemento y las dos matrices deberán tener la misma dimensión.
- ▶ Operaciones matriciales. Hay algunas operaciones usuales en matrices que también podemos realizar en Matlab:
 - La **transpuesta de una matriz**. Se puede emplear el bloque `transpose` o el bloque de `Math Function` dentro de *Math Operations* configurado en la operación *transpose*.
 - La **inversa de una matriz**: Para calcular la inversa de una matriz deberemos introducir un bloque de producto y configurarlo de la forma adecuada. Esto es, primero deberemos introducir el símbolo `"/"` en la opción *number of inputs*. Esto indicará que queremos obtener el inverso de la entrada. A continuación deberemos indicar a Simulink que queremos realizar el producto como producto de matrices.

Cuando trabajamos con matrices y vectores también necesitamos acceder a alguno de los elementos de una matriz y combinar matrices y/o vectores para generar nuevos elementos. Estas acciones las realizaremos con los siguientes bloques:

- ▶ **Concatenación de matrices**: gracias a los bloques `Vector Concatenate` y `Matrix Concatenate` podemos unir dos vectores o matrices. Ambos bloques son equivalentes y se pueden intercambiar en su configuración. Si estamos bajo el modo de

concatenación de matrices podemos indicar la dimensión en la que queremos concatenar bajo el ítem *concatenate dimension*.

- ▶ **Intercambiar dimensiones:** En algunas situaciones nos interesará intercambiar dos dimensiones de una matriz o hipermatriz. Esto nos permitirá realizar una reordenación de los elementos. Podemos realizarlo a través del bloque *Permute Dimensions*. En una matriz de dimensión 2 realizará el mismo efecto que la transposición de matrices.
- ▶ **Seleccionar uno o más elementos de una matriz.** A menudo tenemos que acceder a uno o algunos elementos de una matriz o vector. Podemos realizar esta acción empleando el bloque *selector* que encontraremos bajo la pestaña *Signal Routing*. Este bloque es un bloque muy polivalente con varias configuraciones a tener en cuenta. El primer elemento configurable será el número de dimensiones que tiene la señal de entrada. A partir de este punto, nos aparecerán tantas líneas de configuración como dimensiones tenga la señal de entrada. Para cada dimensión podremos seleccionar una de las siguientes opciones:
 - *Select all*: para seleccionar todos los elementos de la dimensión.
 - *Index vector*: para indicar cuál es el vector de elementos que emplearemos para seleccionar los elementos. La selección se realiza con la misma estructura que la selección por vectores de Matlab.
 - *Starting index*: Permite indicar un índice y el número de elementos que queremos seleccionar.
 - *Starting and ending indices*: Permite indicar el primer y el último índice que queremos seleccionar. Al utilizar esta configuración únicamente tendremos disponible la entrada por puerto.

En las configuraciones que piden los índices podremos configurar en modo *dialog* o en modo *port*. El modo *dialog* nos permite configurar los elementos que vamos a seleccionar a través de la ventana de configuración. En este caso únicamente podremos configurar los puertos de forma estática. Si empleamos el modo *port* podemos configurar los elementos seleccionados a través de una señal de Simulink. Esto nos permitirá realizar la selección de forma dinámica mientras se realiza la simulación.

Alternativamente, en la pestaña *Signal Routing* tenemos el bloque *Index Vector* que nos permite introducir el índice que deseamos en el primer puerto y el vector del que queremos extraer el índice en el segundo.

- ▶ **El multiplexor y el de-multiplexor:** También podemos unir escalares y vectores mediante un bloque *Mux* y deshacer la unión mediante un bloque *Demux*. Tenemos que

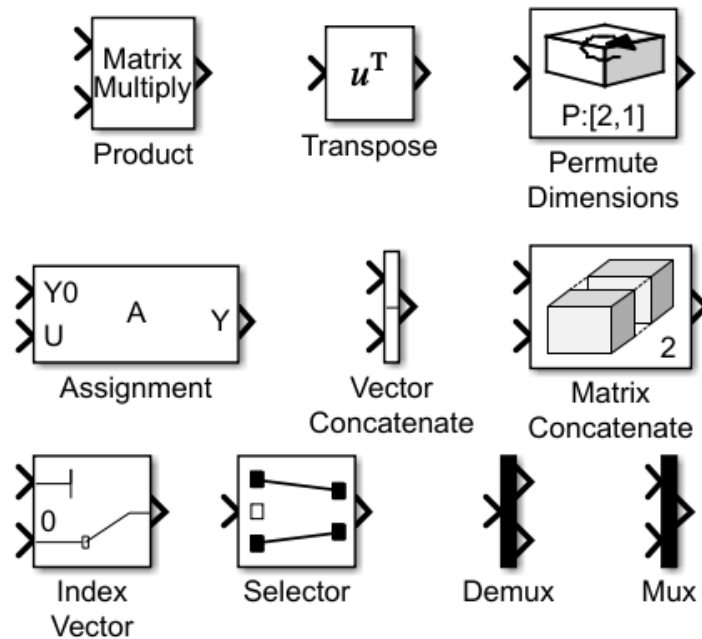


Figura 1: Bloques usados en la vectorización de Simulink.

tener en cuenta que no podremos usar un bloque Mux con matrices. También es importante saber que un bloque Demux dividirá las salidas de forma equitativa entre el número de salidas. Así pues, unimos un escalar con un vector con tres elementos y después lo desunimos con un Demux de dos salidas, obtendremos dos vectores de dos elementos cada uno.

6.3 Agrupamiento de código

El segundo aspecto que deberemos tener en cuenta en este tema es la organización del código. Hemos visto en temas anteriores como los códigos de Simulink pueden complicarse en sobremanera. Esto provoca que sea difícil de seguir y entender el código transcrito. Esto puede suponer un importante contratiempo sobretodo trabajando en equipo.

Para resolver este problema Simulink incorpora la creación de sub-modelos (o sub-sistemas). Un sub-modelo es el equivalente a una función en un lenguaje de programación estándar. De esta forma, podemos generar un bloque que tenga un número de entradas y salidas y efectúe un conjunto de operaciones más complejas. Podremos renombrar dicho bloque y simplificar el diagrama para aumentar la comprensibilidad.

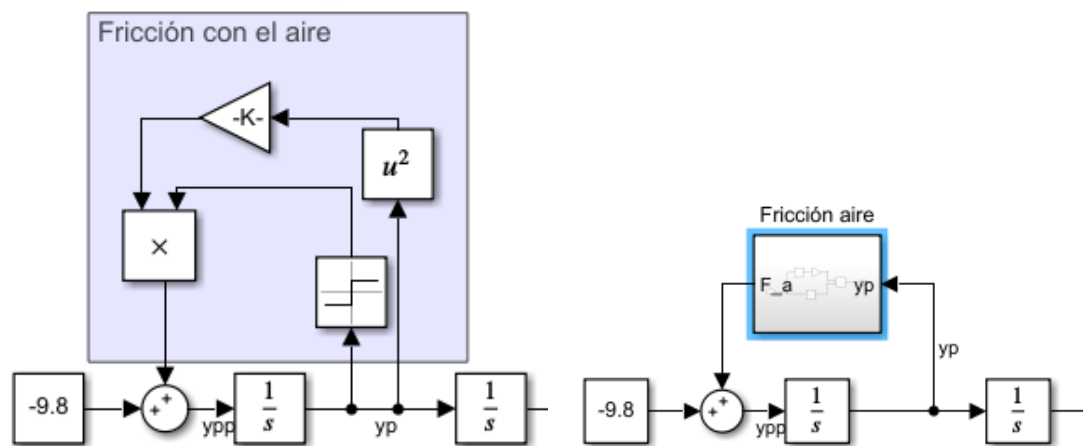


Figura 2: Izquierda: Conjunto de bloques que calculan la fricción con el aire agrupados en un recuadro para clarificar la comprensión del sistema. Derecha: el mismo sistema con los bloques para el cálculo de la fricción atmosférica agrupados en un subsistema.

La primera medida para aumentar la comprensión de un modelo consiste en encuadrar los diferentes bloques para realizar comentarios específicos. Para realizar esta acción deberemos realizar un cuadrado de selección buscar los tres puntos y seleccionar la opción *Create Area*. Una vez hecho esto, nos aparecerá un recuadro (ver Figura 2) en el que todo el código quedará agrupado. Además, podemos poner una anotación sobre la acción que realiza dicho código.

Si bien esta acción ayuda a comprender el modelo, con sistemas muy complejos puede no ser suficiente. El siguiente paso que podemos tomar para aumentar la comprensibilidad de nuestro modelo es juntar un conjunto de bloques en uno solo. En un subsistema.

Tenemos dos opciones para realizar esta acción:

- ▶ Encuadrar el conjunto de elementos que queremos unir en un único bloque y seleccionar *Create Subsystem*. Esto nos generará de forma automática un sub-modelo con todas las entradas y salidas configuradas de la forma adecuada.
- ▶ Introducir un bloque *Subsystem*. Esto insertará un bloque que deberemos programar internamente introduciendo los bloques necesarios.

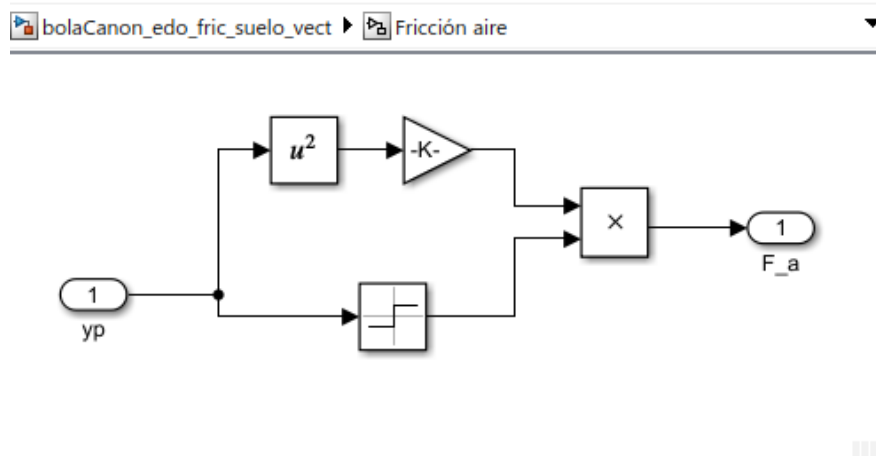


Figura 3: Subsistema generado para representar la fricción con el aire en el sistema mostrado en la figura 2 derecha.

Programación de subsistemas

Una vez introducidos los diferentes elementos dentro de un modelo, deberemos programar la función que le corresponda. Deberemos tener en cuenta los siguientes elementos. El subsistema deberá tener:

- ▶ Al menos un bloque de salida out que emita la señal al resto del sistema. Dicha salida deberá contener el resultado de la simulación del sistema. En algunas situaciones contadas podemos prescindir del bloque de salida si el subsistema se emplea únicamente para escribir o comunicar información al exterior.
- ▶ Un bloque o más de entrada input que recibe la señal del resto del sistema. Deberemos incluir el bloque siempre que tengamos un subsistema que tenga una entrada. No incluiremos este tipo de entrada cuando el subsistema lo usemos para generar una señal con ciertas condiciones de entrada.
- ▶ Podemos introducir el resto de elementos de forma usual para completar el subsistema y obtener el resultado deseado.

La figura 3 nos muestra el recorte de pantalla del subsistema para calcular la fuerza ejercida F_a por la fricción dada una velocidad $\dot{y}$. Podemos ver un bloque de entrada y un bloque de salida. También podemos ver en la parte superior que nos indica que se trata del subsistema “Fricción aire” dentro del modelo de la bola del cañón. A través de esta barra podremos navegar por los diferentes subsistemas que hayamos generado.

Tipos de subsistemas

En la sección anterior hemos introducido la programación general de subsistemas. En algunas situaciones nos interesará ejecutar un sistema únicamente en ciertas situaciones (por ejemplo, cuando suceda un cierto evento o mientras sucede un evento). También podemos incorporar bloques que se activen si se verifica una condición o que repitan un número de veces una condición dada. Veamos los distintos tipos especiales de subsistemas a los que tendremos acceso:

- El **subsistema desencadenado** (*trigger*). Este subsistema se ejecutará únicamente cuando la señal de *triggering* (que se recibe por defecto en la parte superior del bloque) modifique su comportamiento. Podemos configurar qué tipo de comportamiento queremos detectar a través de la configuración *trigger type*, podremos seleccionar:
 - Crecimiento (*rising*): El bloque se ejecuta cuando la señal de *triggering* empiece a crecer.
 - Decrecimiento (*falling*): El bloque se ejecuta cuando la señal de *triggering* empiece a decrecer.
 - Ambos (*either*): El bloque se ejecuta cuando la señal empieza a crecer o a decrecer indistintamente.

En este caso, el bloque únicamente se ejecuta cuando se produce el cambio, no siempre que sea creciente o decreciente. También se puede generar un subsistema de tipo *trigger* añadiendo el bloque con el mismo nombre en el subsistema. Este bloque se puede encontrar en la pestaña *Ports & Subsystems*.

- El **subsistema habilitado** (*enabled*). Este tipo de subsistema se ejecutará siempre que la condición de habilitación esté activa. El subsistema tendrá una entrada en la parte superior que nos permitirá transmitir una señal booleana. Cuando sea afirmativa el bloque se ejecutará. Si es negativa, entonces el bloque no se activará. En la configuración podremos escoger qué valor tendrá la señal de entrada del sistema cada vez que se active. Para ello realizaremos una doble pulsación en el bloque *enable* y modificaremos la configuración *States when enabling*, que podrá ser:
 - Mantener último valor (*held*). Esta configuración emitirá la señal emitida en la última ejecución.
 - Resetear al valor inicial (*reset*). Esta configuración permitirá inicializar de nuevo la variable al primer valor introducido al subsistema.

Como en el caso de los sistemas *trigger* podemos transformar un sistema en sistema *enable* añadiendo el bloque con dicho nombre. También es importante tener en cuenta que podemos añadir ambos bloques a un subsistema para tener un sistema que se ejecute siguiendo las dos condiciones.

► **Subsistemas condicionales.** Los subsistemas condicionales son aquellos que se ejecutan como resultado de un condicional. Tendremos dos tipos de subsistemas:

- Subsistemas condicionales “si..., si no ...” (if, else).
- Subsistemas de tipo “switch”.

Estos bloques se ejecutarán únicamente cuando reciban una señal lógica verdadera de un bloque condicional if o un bloque Switch case. Debemos tener en cuenta que la señal de un bloque if con la alternativa *else* activada será verdadera cuando falle al condición del bloque if.

Los bloques lógicos admiten las configuraciones siguientes:

- El bloque if podemos configurar:
 - el número de entradas del bloque bajo el parámetro *number of inputs*.
 - la condición a verificar bajo el parámetro *if expression*.
 - un conjunto de condiciones que añadiremos para encadenar en bloques *el-seif*. Cada condición deberá ir separada por una coma y nos añadirá un puerto de salida adicional.

También podremos marcar si queremos que haya una salida para la condición “si no” o no.

- En el bloque Switch case solo podemos configurar los valores que tomará el *switch*. Deberemos separar por comas cada una de las posibles salidas. Matlab truncará la señal de entrada y emitirá una señal verdadera a través de la salida correspondiente y falsa en el resto. Si activamos la casilla de la salida *default* también se generará una salida adicional que nos permitirá dar por válida si no hay ninguna salida coincidente.

► **Subsistemas iterativos tipo for:** Este tipo de subsistema nos permite repetir las operaciones el número designado de veces en el bloque for bajo el ítem: *iteration limit (N)*. El resto de parámetros funcionan igual que en los bloques anteriores.

El bloque for tiene una única salida para indicar el índice del bucle. Esto nos puede interesar si el índice es importante en la iteración.

► **Subsistemas iterativos tipo while:** Este tipo de subsistema sustituye a los bucles *while* y *do-while*. El bucle se repetirá mientras se dé la condición estipulada en

el bloque `while`. El subsistema se ejecutará de forma ininterrumpida mientras se reciba una señal verdadera en la entrada del bloque `while` o hasta que se alcance un número máximo de iterados definido en el ítem *Maximum number of iterations*. Podemos fijar a un número indefinido de iterados introduciendo el valor -1 en este último parámetro.

Además, podremos escoger entre dos tipos de iteraciones:

- iteración tipo *while*. Se ejecuta siempre que la condición sea verdadera. Si la primera vez la condición es falsa, no se ejecutará.
- iteración tipo *do-while*. La primera ejecución se realiza siempre. Al final de la iteración se verifica el valor y se determina si se para.

Como es habitual en cualquier entorno de programación, se podemos guardar un subsistema en un archivo independiente. Dicho fichero deberá contener sus entradas y salidas de la forma usual. La ventaja que tendremos es que podremos usarlo en varios proyectos.

El último concepto clave sobre los subsistemas es el modo de ejecución. Por defecto, cuando ejecutamos un sistema que tiene algún submodelo, Simulink despliega el submodelo y realiza la ejecución. En algunas situaciones nos interesará evitar esto y hacer que el sistema se ejecute de forma independiente. En este caso, deberemos indicar a Simulink que se trata de un subsistema atómico en los parámetros del bloque del subsistema.

Ejercicio 1. Elabora un subsistema que calcule la fuerza ejercida por la fricción con el aire siguiendo la fórmula:

$$\vec{f} = -\frac{1}{2}\rho AC_D \|v\| \vec{v}$$

El subsistema recibirá una única entrada en forma de vector con las tres coordenadas de la velocidad y una única salida también en forma de vector con las tres coordenadas de la fuerza.

Para resolver este problema vamos a insertar un nuevo subsistema en Simulink. Podemos realizarlo de tres formas distintas:

- introduciendo un subsistema des de la biblioteca de bloques.

- ▶ Creando el subsistema en el modelo principal y seleccionando aquellos bloques que forman el sistema.
- ▶ Creando un nuevo subsistema desde el desplegable nuevo.

Vamos a optar por la tercera de las opciones. Así, tendremos un bloque generado para representar la fricción con el aire que podremos introducir en cualquier modelo.

Una vez tengamos el modelo vacío vamos a necesitar los siguientes bloques:

- ▶ Un bloque `input` (contendrá el vector de velocidades del objeto en movimiento).
- ▶ Un bloque `output` (dará como salida el vector correspondiente a la fuerza ejercida por la fricción con el aire).
- ▶ Un bloque de transposición `transpose` junto con un bloque de producto de matrices (bloque `product` configurado en modo *Matrix (*)*) y un bloque de raíz cuadrada `Sqrt`. Estos tres bloques nos permitirán calcular la norma del vector de entrada mediante la fórmula:

$$\|v\| = \sqrt{v^T \cdot v}$$

- ▶ Tres bloques `constant` para almacenar las constantes ρ , A y C_D .
- ▶ Un bloque `gain` para cambiar el signo del vector configurado a -1 .
- ▶ Un bloque de producto, configurado en *element-wise* para realizar el producto de las constantes, el vector y la norma del vector que hemos calculado.

Una vez incorporados todos los bloques a nuestro subsistema, realizamos la conexión tal y como se muestra en la Figura 4. Una vez finalizado, guardaremos el modelo con el nombre `friccionatmos`.

Observación: En la construcción del modelo, podríamos poner las tres constantes dentro del mismo bloque `Gain` igual que hicimos en los ejercicios de los temas anteriores. Sin embargo, lo realizaremos así para dejar el sistema preparado para la realización del ejercicio 2.

Esta implementación que acabamos de realizar se corresponde con una implementación realista de la fricción ejercida por un fluido. A diferencia de la aproximación que realizamos en ejercicios parecidos anteriores. Ahora nos queda incorporarla a nuestro

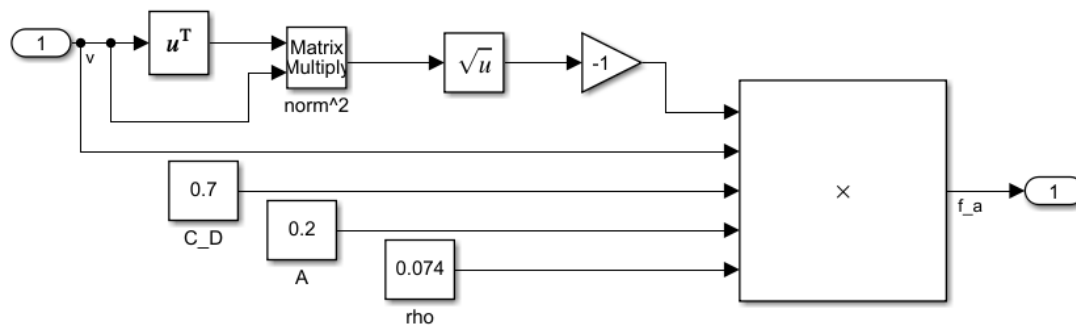


Figura 4: Subsistema que calcula la fricción del aire dado el vector velocidad.

modelo. Para ello, partiremos del modelo generado en el ejercicio 1 del tema 5. Vamos a eliminar toda la parte central que se corresponde con el cálculo de la fricción atmosférica y la sustituiremos por nuestro nuevo bloque. Para ello realizaremos:

1. Insertaremos un bloque `Subsystem` `reference` que encontraremos en la pestaña *Ports & Subsystems*.
2. Realizaremos una doble pulsación sobre el bloque para abrir la configuración.
3. Usaremos el botón *Browse* y buscaremos el subsistema `friccionatmos`.

Ahora solo nos queda unir las dos velocidades para formar un vector con un bloque `Concatenate Vector` que podemos encontrar en la pestaña *Math Operations* y separar las fuerzas con un bloque `Demux` que encontraremos en la pestaña *Signal Routing*. El resultado del sistema viene definido en la Figura 5.

Si comparamos los resultados con los que obtuvimos en el modelo con la fricción implementada por componentes del tema 4, tenemos que la bala recorre 65.25m comparados con los 70.71m que obtuvimos en el caso anterior, y ha tardado 3.893s en vez de los 3.922s que tardó en el ejemplo anterior. Así pues, podemos apreciar una diferencia notable en el resultado del modelo. La figura 6 muestra la comparación en el recorrido de la bala a lo largo del tiempo.

6.4 Usando Simulink desde Matlab

El último aspecto que nos queda por ver es cómo ejecutar Simulink desde el Workspace de Matlab. Esto nos resultará especialmente útil para incorporar nuestros modelos

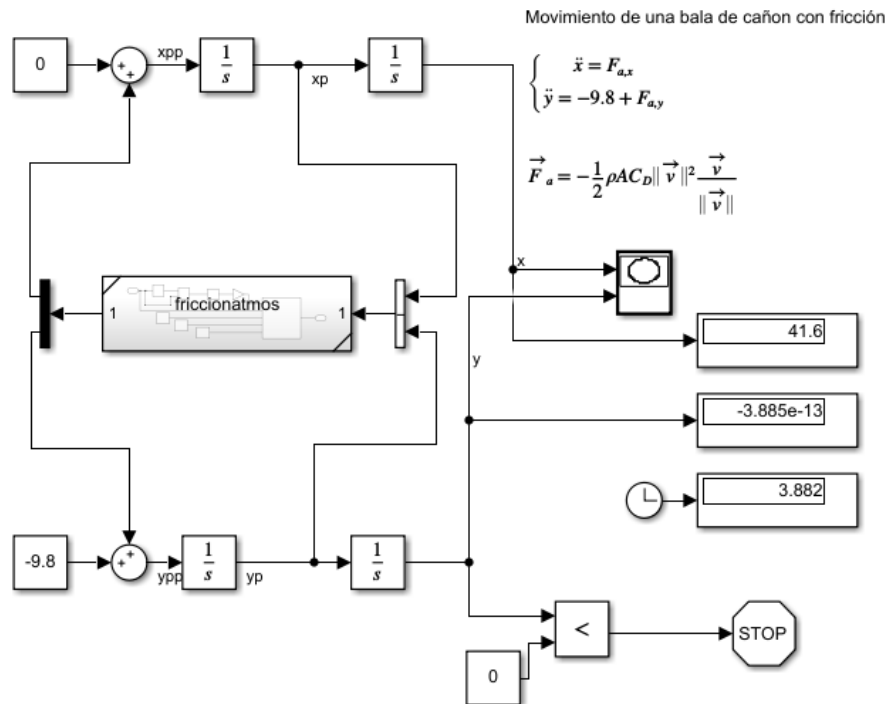


Figura 5: Modelo para una bala de cañón sujeto a la fricción con el aire usando el subsistema de la fricción con el aire.

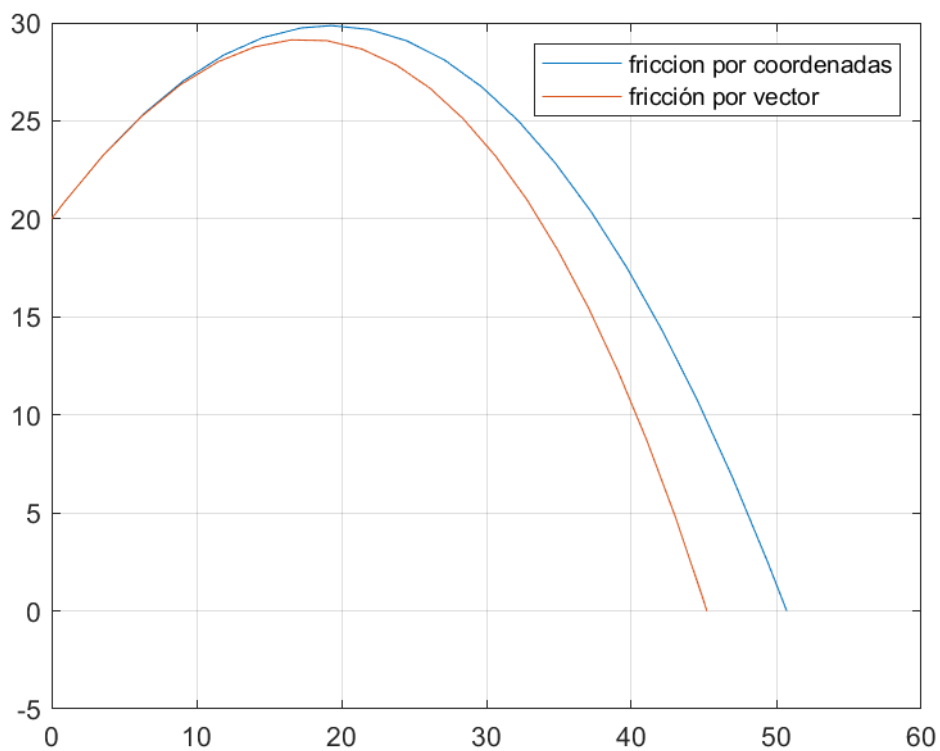


Figura 6: Resultado del vuelo de una bala de cañón calculado con la fricción tomada como un vector o la fricción tomada coordenada por coordenada.

a programas y funciones que hayamos elaborado en Matlab. La instrucción elemental viene dada por el comando:

```
sim('nombre_modelo')
```

Cuando utilizamos este método de llamada a un modelo de Simulink, se utilizan todos los parámetros por defecto. Vamos a ver cómo modificar el modelo para que podamos modificar parámetros a partir del *Workspace* de Simulink.

El primer punto que tenemos que tener en cuenta es que por defecto, todos los modelos en Simulink tienen acceso a las variables definidas en el Workspace de Matlab. Con esto, podremos incorporar cualquier valor de cualquier bloque mediante el nombre correspondiente de la variable de Matlab.

Si realizamos la llamada del modelo a través de una función, deberemos indicarle a Simulink que debe buscar las variables en el espacio de trabajo de la variable y no en el Workspace de Matlab. Haremos esto con la instrucción:

```
options = simset('SrcWorkspace','current');
```

En cuanto a las salidas de los sistemas, cuando empleemos la función `sim matlab` nos devolverá una estructura de datos con la salida de la simulación. Esta estructura contendrá:

- ▶ una variable `tout` con los tiempos de los pasos de integración.
- ▶ Una variable con las señales recibidas por los bloques `out` que contenga el sistema.
- ▶ Una variable por cada bloque `output to Workspace`.
- ▶ Una estructura de datos con los detalles de la última compilación.

En el vídeo “llamando a Simulink desde Matlab” profundizaremos en las diferentes opciones que podemos emplear para llamar a Simulink a través de Matlab. Veremos tanto como realizar la llamada como un estudio exhaustivo de las diferentes señales que saldrán del sistema.



Otro aspecto que podemos mejorar, es relativo a las condiciones iniciales. Hasta ahora, sabemos como inicializar las condiciones iniciales a través del bloque *Integrate*. Sin embargo, en algunas situaciones, la condición inicial dependerá de ciertas entradas al sistema. Así pues, deberemos indicar a Simulink que queremos realizar algunos cálculos antes de inicializar la variable. Por suerte, Simulink nos permite entrar la condición inicial mediante un puerto especial. Para ello, abriremos al configuración del bloque y cambiaremos el desplegable *Initial Condition source* a la opción *external*. Esto nos permitirá obtener un nuevo puerto de entrada del sistema que nos permitirá introducir los valores de la condición inicial.

Ejercicio 2. Realiza los cambios necesarios al modelo del ejercicio 1 para tener en cuenta los siguientes cambios:

- ▶ El sistema tendrá definidas las constantes siguientes en el Workspace de Matlab:
 - La altura del cañón h .
 - El ángulo de disparo del cañón θ .
 - La velocidad de disparo del cañón v_0 .
 - El área sometida al viento A .
 - La presión atmosférica ρ .
 - El coeficiente aerodinámico C_D .
- ▶ El modelo deberá calcular la velocidad en cada una de las dos componentes del sistema y pasarlas como condición inicial de la posición y la velocidad respectivamente.

Ejecuta el modelo en Matlab con los parámetros $h = 25\text{m}$, $\theta = \pi/6$, $v_0 = 30\text{ms}^{-1}$, $A = 0.3\text{m}^2$, $\rho = 0.074\text{kgm}^{-3}$ y $C_D = 0.5$ y dibuja la gráfica del recorrido realizado por la bala del cañón.

Para realizar este ejercicio vamos a introducir los elementos necesarios que nos solicita el enunciado:

- ▶ Para fijar la altura del cañón, h , la definiremos con la constante `h` directamente

como condición inicial para la altura y . Esto lo haremos directamente en el bloque `integrate` que define la integración de y .

- ▶ Para el cálculo de la velocidad inicial en cada componente usaremos los parámetros θ y v_0 los vamos a introducir como constantes en el modelo principal. Los configuraremos con los nombres correspondientes a las variables definidas: `theta` y `vel0`. Vamos a necesitar un bloque de función trigonométrica en modo *sincos* para calcular el seno y el coseno del ángulo θ . También necesitaremos un bloque `product` para multiplicar por la velocidad inicial v_0 . También configuraremos los bloques de integración de $\ddot{x}$ y $\ddot{y}$ con la condición inicial de forma externa y conectaremos los valores correspondientes a la velocidad inicial en cada dirección.
- ▶ Los parámetros A , ρ y C_D los configuraremos con los nombres `area`, `rho` y `coefD` en los bloques constantes que hemos dejado preparados del ejercicio 1 del subsistema `friccionatmos`.
- ▶ para extraer los datos usaremos un bloque `To Workspace` configurado en modo *Save Format: Array*. La variable de salida será `estados` y contendrá un vector creado a partir de la concatenación de las señales del tiempo, la posición en x y la posición en y .

La figura 7 muestra como debemos realizar las conexiones del modelo y del subsistema de fricción con el aire.

Tras finalizar estos pasos, veremos que los nuevos bloques que hemos definido como variables aparecen marcados en rojo. Esto nos indica que estas variables no están definidas en el *Workspace* de Matlab y, por lo tanto, el sistema no funcionará. Debemos acudir a la ventana de comandos de Matlab e introducir las distintas variables con los elementos inicializados según las especificaciones del sistema.

Una vez introducidas las variables, podemos ejecutar el modelo de simulación. El modelo lo hemos guardado con el nombre `bolaCanon_t6e2`. Guardaremos el resultado de la simulación en la variable `res`. Finalmente extraeremos los valores de las variables en Matlab para realizar la representación gráfica. El siguiente código generará las variables necesarias, ejecutará el modelo, extraerá las variables y generará la gráfica.



```
%definimos variables del sistema
```

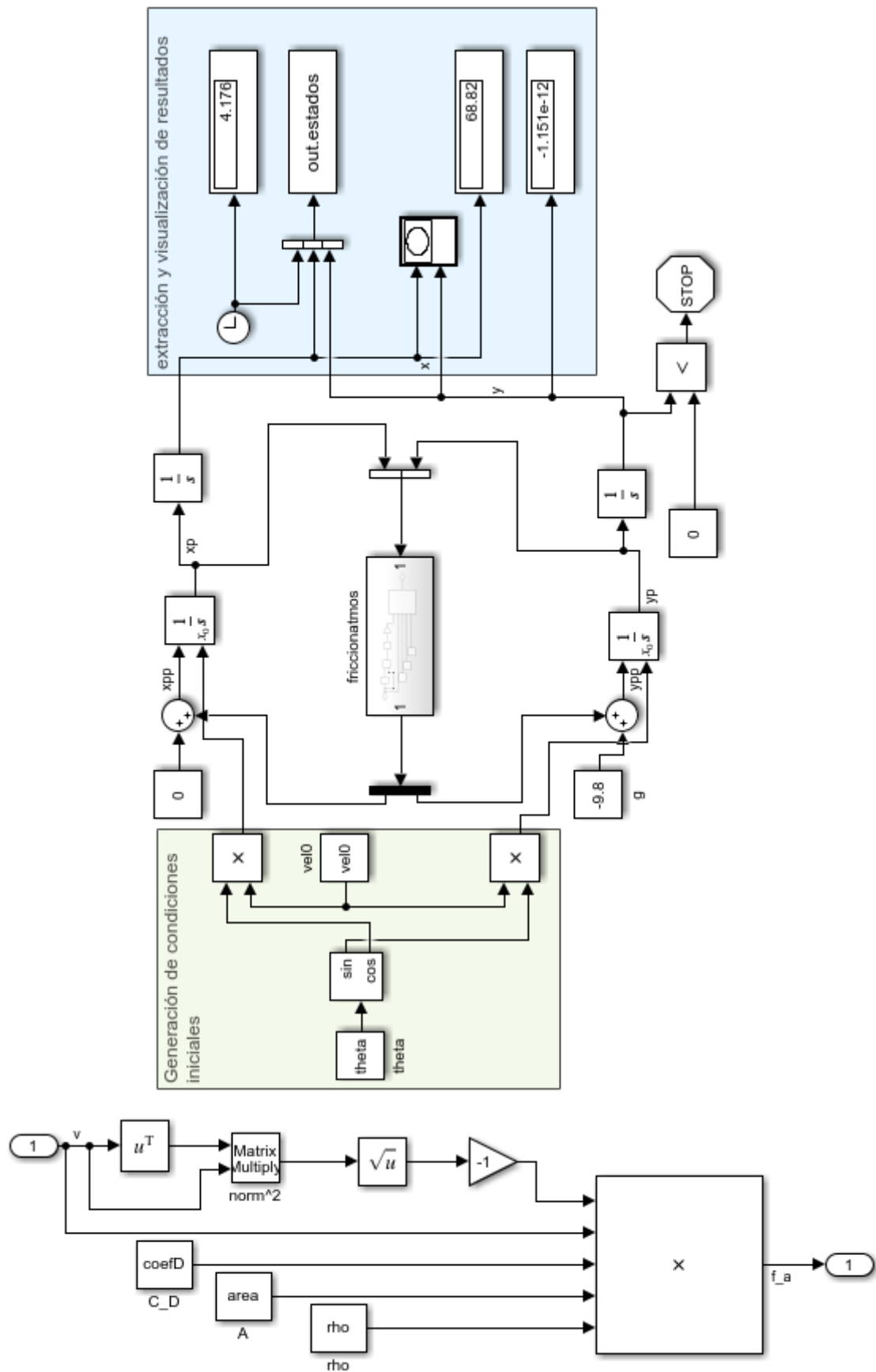


Figura 7: Modelo en Simulink para el cañón con la introducción de los parámetros solicitados en el ejercicio 2.

```

h=25;
theta=pi/6;
vel0=30;

area=0.3;
rho=0.074;
coefD=0.5;

%ejecutamos el sistema
simout=sim('bolaCanon_t6e2.slx');

%recuperamos el vector de estados
estados=simout.estados;

%realizamos la gráfica de la posición contra el tiempo
plot(estados(:,2),estados(:,3))
xlabel ("x")
ylabel("y")
grid on

```

En el vídeo “El modelo de la bala de cañón ampliado” vamos a ampliar a la tercera dimensión el modelo que hemos generado para la bala de cañón. Tendremos en cuenta el movimiento en las tres dimensiones y el efecto de un choque no elástico con el suelo.



Accede al vídeo: El modelo de la bala de cañón ampliado